

RESEARCH ARTICLE

Enhancing Edge-Linked Caching in Information-Centric Networking for Internet of Things With Deep Reinforcement Learning

HAMID ASMAT¹, IKRAM UD DIN¹, (Senior Member, IEEE),
AHMAD ALMOGREN², (Senior Member, IEEE), AYMAN ALTAMEEM³,
AND MUHAMMAD YASAR KHAN⁴

¹Department of Information Technology, The University of Haripur, Haripur 22620, Pakistan

²Department of Computer Science, College of Computer and Information Sciences, King Saud University, Riyadh 11633, Saudi Arabia

³Department of Natural and Engineering Sciences, College of Applied Studies and Community Services, King Saud University, Riyadh 11633, Saudi Arabia

⁴SFI-Funded E-Governance Unit, Insight Centre for Data Analytics, University of Galway, H91 TK33 Ireland

Corresponding author: Muhammad Yasar Khan (muhammadyasar.khan@universityofgalway.ie)

This work was supported by King Saud University, Riyadh, Saudi Arabia, through Researchers Supporting Project RSP2024R498.

ABSTRACT This paper proposes an Enhanced Edge-Linked Caching (EELC) scheme for Internet of Things (IoT) environments under Information-Centric Networking (ICN), employing an advanced use of Proximal Policy Optimization (PPO), a form of deep reinforcement learning, to inform the caching decisions of edge nodes. The rapid proliferation of IoT devices has led to significant challenges in managing content efficiently within networks, particularly in terms of scalability, latency, and energy consumption. Traditional IP-based architectures are inadequate in addressing these challenges, necessitating a shift towards a content-centric approach provided by ICN. By leveraging PPO, EELC dynamically adapts to changing IoT network conditions, optimizing caching strategies to enhance energy efficiency and improve network responsiveness. In our simulation, we verify the performance of EELC in comparison to the Edge-Linked Caching (ELC) and Leave Copy Everywhere (LCE) approaches under different cache sizes and Zipf-distributed content requests. EELC performs far better than ELC under energy efficiency, cache hit ratios, and server hit reduction in all tested scenarios. This indicates that EELC could be a potential approach for significantly improving network efficiency and responsiveness in IoT networks.

INDEX TERMS Information centric network, caching, Internet of Things, content update, machine learning, reinforced learning, proximal policy optimization.

I. INTRODUCTION

The Internet of Things (IoT) represents a significant evolution in the realm of networking and computing, bringing forth a vast network of interconnected devices capable of generating, processing, and transmitting copious amounts of data. From basic sensors to complex systems, these devices play a pivotal role in the constantly expanding digital ecosystem. This broader network domain calls for extreme security provided with strong protective measures against threats and vulnerabilities [1]. Content management and network

efficiency face several challenges due to increase in the number of IoT devices and their contents [2].

However, classical IP-based network architectures are revealing their inappropriateness to cope with the enormous scale and complexity intrinsic to IoT networks. Such characteristics of environments typical for the IoT are, thus, not compatible with the centralized way that IP networks work. This inadequacy is perhaps particularly exposed by the challenges of content forwarding and content caching, and thus there is a need for the reassessment of conventional networking paradigms [3].

The aforementioned issues were resolved by introducing Information Centric Network (ICN) in [4]. The ICN model

The associate editor coordinating the review of this manuscript and approving it for publication was Petros Nicopolitidis.

is a move from a host-centric or IP-based network to a content-centric model. It puts much emphasis on naming content more than hosts, and such an approach perfectly fits the content-centric nature of IoT networks. Such a move is beneficial when offering efficiency to content management and dissemination across IoT devices and their content, forming a highly robust framework in support of the requirements from the ecosystems of IoT [5]. Caching in ICN acts as an important way of enhancing the efficiency of a network by storing contents near users reducing latency and bandwidth utilization in IoT applications [6]. Figure 1 shows ICN caching, where a single request is forwarded to the publisher, and the content is provided directly by the publisher. For subsequent requests, the content is served by different cache nodes. While several methods have been proposed to cache IoT content effectively, but optimizing cache performance in dynamic IoT environments still faces challenges. Machine learning offers a major tool in addressing these challenges in the realization of smarter, context-aware caching strategies. Reference [7] focuses on the machine learning integration into ICN-caching design for IoT environments to provide an insight into its necessity and efficiency. The advantages of utilizing machine learning with ICN caching for IoT environments are manifold. It allows smart decision-making in the caching and distribution of content, resulting in better network efficiency and user experience. References [8] and [9] explored various machine-learning-based routing and caching schemes, presenting potential research directions to improve ICN performance in IoT settings. Building on these advancements, deep reinforcement learning (DRL), a subset of machine learning, further enhances these caching strategies by combining decision-making processes with a reward-based learning mechanism. DRL systems learn optimal caching decisions by interacting with the network environment, using feedback to adapt and improve over time. This adaptability makes DRL well-suited for managing dynamic and unpredictable IoT traffic, leading to better resource utilization, reduced latency, and improved overall network performance. DRL is a machine learning technique where an agent learns to make optimal decisions by interacting with its environment and receiving feedback in the form of rewards. This approach enables the agent to continually improve its performance in complex and dynamic scenarios [10].

Another major achievement in caching schemes for IoT environment is the ELC (Edge-Linked Caching) scheme. ELC has been designed to reduce energy consumption, while also reducing the load on IoT nodes, by minimizing access requests and hosting content closer to the subscribers at the leaf nodes of the network. This scheme has a Path-table located at the edge node, which logs paths for the requests of content, as well as the information related to where the content is located. This table is then used by the edge node in content management updates and caching over the network so that content can be made available wherever it might be needed without unnecessary

duplication or transmission. Moreover, this doesn't only ensure energy efficiency among IoT nodes but also improves general efficiency and responsiveness in content delivery upon demand. As the content is cached at the leaf node closest to a subscriber and relies on the Path-table for content management, ELC offers an excellent improvement over traditional methods of caching, especially in cases where content is transient and needs to be updated periodically [11].

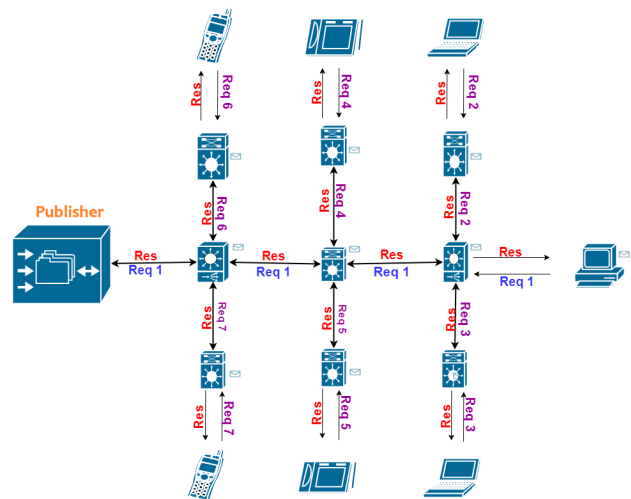


FIGURE 1. Caching in ICN.

This work builds upon the ELC scheme by adopting the Proximal Policy Optimization (PPO) algorithm [12], a DRL technique, to refine and enhance caching performance. The integration of PPO aims to improve energy efficiency, increase cache hit rates, and reduce server loads by enabling more context-sensitive decision-making in the caching process, resulting in the design of the Enhanced Edge-Linked Caching (EELC) scheme. In this way, we seek to leverage the strengths of ELC while addressing its limitations, paving the way for more sophisticated and effective mechanisms in ICN-based IoT networks.

This study presents several key contributions, described as follows:

- This study points out the use of advanced machine learning methods to make data storage more efficient in the Internet of Things.
- It shows how PPO algorithm can help reduce energy consumption and server resources in complex IoT systems.
- It also shows evidence that the PPO is particularly effective in how quickly data fetches are successful at first try, which proves its worth when managing IoT networks.

The rest of the paper is structured as follows. Section II covers existing caching schemes in the ICN-IoT environment. Enhancements to edge-linked caching are presented in Section IV. Section V explains the simulation environment,

and Section VI presents the results and analysis, while the conclusion is provided in Section VII.

II. RELATED WORK

The importance of Machine Learning (ML) for the growth of ICN—especially in IoT settings—is pivotal because of its inherent capacity to handle large data sets and optimize network operations instantly. ML's pattern recognition and predictive capabilities are useful in environments where content is of primary importance [7].

A. ICN-IoT SECURITY

IoT security is improved by introducing ML in the ICN-IoT. Due to its self-adjusting nature, ML can detect new threats quickly and take appropriate steps to mitigate risks by analyzing different patterns in the network. This makes it an efficient tool to reinforce ICN-IoT systems' security [13].

B. ICN-IoT MOBILITY

ML is vital for mobility control in IoT networks based on ICN. ML techniques can predict node movement to help improve data routing and content delivery efficiency, which further enhances the performance of networking and the user experience in dynamic setups [14], [15].

C. ICN-IoT CACHING

ML has significant benefits in enabling in-network caching for ICN. It is crucial to optimize caching strategies by analyzing and predicting network traffic patterns in ICN-based IoT systems. Improved caching directly enhances network performance and efficiency [16], [17] highlight the role of ML in refining caching mechanisms, leading to more efficient IoT networks.

ML is an emerging technology that brings significant improvements to ICN, improving many aspects of IoT networks, particularly in content caching. Its predictive capabilities ensure efficient use of network resources, improved security, and an enhanced user experience. Machine Learning is an important aspect in enhancing Information-Centric Networking environments, specifically issues in dynamic content caching. Below, we review different scheme which uses machine learning for caching contents in the network.

In IoT environments, caching is becoming increasingly important. It is key for faster data transfers and reduces the load on networks. Multiple interconnected devices in an IoT networks generate large amounts of data, which requires robust caching infrastructure to deliver content fast and reliably. Traditional caching methods, primarily designed for more predictable and static content patterns, face challenges in the IoT context. These conventional approaches often struggle to adapt to the highly dynamic and heterogeneous nature of IoT content, which encompasses a wide variety of content types and fluctuating access patterns [18]. Following are the studies that uses machine learning in the context to ICN-IoT caching

ML is changing how caching is handled in IoT networks. It analyzes and optimizes complex data models, enabling it to predict future needs. This is particularly useful in the IoT, where the variety and amount of information available makes traditional methods inefficient. ML models make intelligent decisions by acquiring frequency and recent information, as well as its relevance over time. Moreover, it improves storage efficiency, ensuring that the most important resources are always available. This not only improves network efficiency but also enhances user experience in IoT environments [19], [20]. Following are some articles, using machine learning in ICN-IoT caching.

The importance of integrating communications, storage and computing resources in the context of connected vehicles was emphasized in [19]. The study used a deep reinforcement learning (DRL) approach, demonstrating its usefulness in delay-sensitive applications and highlighting the benefits of edge caching. Furthermore, a method based on deep reinforcement learning (DRL) was introduced in [17] to cache IoT content at network edges. It focused on balancing communication costs and content freshness. This method showed improvements in traditional systems without relying on default content popularity or user request patterns. Moreover, considering [21] transient content characteristics, dynamically adjusting content popularity and access patterns in a DRL-based approach, this method optimizes collection decisions, by its adaptability, enables cache resource management and it may restrict its use in constrained environments. A new aggregation scheme using data classification in NDN [22] was proposed, which focused on adapting popular dynamic features, which improved network performance. this contribution for Information Centric-Internet of Things (IC-IoT) architecture was based on software-defined networks. In another instance, [23], utilized deep Q-learning to manage computing and caching resources. This new approach maximized rewards by aggregating popular AI models, considering both immediate and long-term benefits. Furthermore, DeepCache [24], which is a caching scheme based on LSTM Encoder-Decoder model. This approach improved cache hit rates when integrated with existing caching policies, highlighting the effectiveness of LSTM models in predicting future content characteristics. Furthermore, a graph neural network-based (GNN) caching scheme in a named data network (NDN) [25] significantly improved cache hit ratios and reduced latency, demonstrating the effectiveness of GNN in determining the content caching probability for each network node. Reinforcement learning (RL) is employed to intelligently manage storage in the context of IoT applications, increasing the flexibility of storage by adapting to changes in context. This scheme aims to ensure the caching of high-quality content, with the expectation that the context will change, thereby increasing the relevance of the content cached. However, a major challenge remains the rely on accurate context predictions, which may struggle with constantly changing environments [26]. Moreover, GNNs implemented in named data networks (NDNs) use

network topology and content query patterns to make caching decisions. This approach predicts the popularity of features, improving network performance. However, the complexity of the GNN framework and the need for detailed network content for training highlights the challenges of implementation in scalable environments and the integration of GNN with deep reinforcement learning [27].

Similarly, DRL provides an alternative to caching in a software-defined networks (SDN) based on ICN. This connectivity allows learning and optimizing network topologies and traffic patterns, increasing collection efficiency. However, the computational requirements and methodological issues highlight the need for further research to address these challenges [28]. In a recent study [29], the authors propose a parameter optimization framework for improving energy efficiency in IoT-based Named Data Networking (NDN). By analyzing factors like packet size and caching strategy, they use hyper-parameter optimization to identify optimal configurations that minimize energy consumption. The systematic analysis provides precise insights, achieving considerable energy savings. The framework's versatility in adapting different parameters makes it useful for diverse network conditions, but the complexity and processing demands involved may be impractical for real-time application, especially for constrained devices with limited computational power.

A novel seq2seq LSTM instances optimize cache performance by prioritizing content reputation and caching. The strength of this model lies in its ability to deal with sequential data, which provides nuanced understanding of content priority. However, it depends on the availability of detailed historical data, and the complexity of the model can pose challenges to real-time implementation [30]. In [31], the workflow for removing edge caching analyzes user actions and provides individualized caching options, of which The goal is to improve the cache hit rate. While promising user satisfaction, this approach requires careful privacy considerations, including the need for efficient registration mechanisms. In another study, the authors propose the iCache framework, which provides an intelligent caching algorithm designed to adapt and dynamic states of IoT networks based on ICN through machine learning algorithms using it to determine the popularity of content and interactive environments to enhance the user experience. The disadvantage is that it is difficult to ensure that the data used to make informed caching decisions is accurate and timely [32].

Furthermore, [33], a Deep Reinforced Learning is utilized for caching and evicting transient content, which dynamically regulates cache decisions by using content popularity, size, and freshness, thus aiming to improve cache utilization and minimize content retrieval times by forecasting user demand and adapting to network changes. The advantage of this methodology is its capacity to improve cache utilization and efficiency, which translates to enhanced network performance and diminished latency. Conversely, its reliance on

historical data for predictions might not always accurately reflect future demands in swiftly evolving or new network landscapes. Moreover, in [34], the authors have designed an optimal caching strategy for edge computing in IoT, focusing on maximizing data freshness using a Markov Decision Process (MDP). This strategy reduces energy consumption and cache miss rates, ultimately extending node longevity. The use of Age of Information (AoI) allows the system to ensure the timeliness of data, resulting in improved Quality of Service (QoS). However, the computational resources required for solving MDPs can be considerable, especially in environments with a high volume of IoT nodes, which may lead to potential performance bottlenecks and limited real-time applicability.

In [35], a caching scheme for IoT setting, which is a clustering approach that combines deep reinforcement learning (DRL) and Markov Chain Monte Carlo (MCMC). This approach seek to balance content availability and optimize energy consumption by modifying caching decisions according to user needs and network conditions. Additionally, to shape future storage decisions DRL learns from historical data, while MCMC helps inform optimal policy caching schemes. Main benefit of this scheme lies in balancing increased cache utilization and reducing power consumption, which is an important consideration for energy constraint IoT devices. However, challenges remain in integrating DRL and MCMC, particularly with resource limited availability. Furthermore, a novel application of software-defined networking (SDN) for resource scheduling across heterogeneous edge computing networks is discussed, which capitalizes on reinforcement learning for dynamic resource allocation. This scheme aims at elevating edge computing services' performance and reliability through optimal computing and storage resource distribution. Its advantage is the improvement in resource utilization efficiency it brings, guaranteeing superior service quality. However, its scalability poses a challenge, as it demands considerable computational resources for continuous network adaptation, potentially limiting its efficacy with network expansion [36].

Moreover, VeSoNet introduces a deep reinforcement learning-based content caching scheme for vehicular social networks, considering the unique challenges of high mobility and variable network density. By integrating path planning and social interaction content to forecast content requests, it adaptively revises its caching strategy to ensure content is available when needed. This approach effectively handles the dynamic nature of vehicular networks, ensuring high cache hit rates and efficient content delivery. However, the need for frequent retraining of the DRL model, due to rapid shifts in network conditions and vehicle mobility, may restrict its long-term effectiveness [37].

Similarly, the authors of [38] propose an Intelligent Edge Caching (ICE) scheme using DRL to enhance caching efficiency by capturing valuable data insights and employing a popularity model based on Newton's law of cooling. The

scheme improves caching decisions using a Markov decision process, and further introduces DCCC, a distributed multi-node caching framework assisted by a multi-tiered caching hierarchy. The single-node ICE and DCCC outperform existing methods in cache hit rate and caching efficiency. ICE improves adaptability and cache utilization, while DCCC optimizes multi-node caching, leading to better resource management. However, increased complexity and potential overhead limit scalability, and real-world validation is needed beyond simulations to fully assess practical benefits. The approach may also require significant computational resources, which could be a challenge for edge devices. [39], introduce a Q-learning-based forwarding strategy for Information-Centric Networks (ICN), aiming to minimize data retrieval latency and network congestion. Their approach uses Q-learning to optimize forwarding paths, resulting in improved network throughput and reduced server load. This adaptive strategy allows for more efficient routing in varying traffic conditions, enhancing overall network performance. However, the reliance on Q-learning, which depends heavily on historical data, can lead to slower adaptation in rapidly changing environments, and the increased learning period may also affect real-time decision-making capabilities.

In [40], the authors propose energy-efficient content fetching strategies for cache-enabled device-to-device (D2D) networks using an actor-critic deep reinforcement learning (DRL) framework. The approach aims to minimize collision rates and optimize throughput, leveraging a two-level actor-critic algorithm to adjust content fetching strategies dynamically. By optimizing the network architecture and reward function, the method effectively balances collision rates and energy consumption through weighted reward values. This strategy significantly reduces content fetching collisions compared to baseline algorithms and enhances energy efficiency. The proposed solution dynamically adapts to network conditions and reduces energy usage; however, it may have increased implementation complexity and require high computational resources, potentially limiting its applicability in resource-constrained environments. In another study [41], the authors present a transient data caching strategy using a maximum entropy Actor-Critic (TD-MEAC) method to enhance the freshness of cached data in IoT edge networks. The proposed approach effectively manages transient data and reduces long-term caching costs. Simulation results indicate enhanced cache hit rates and data freshness. The strength of this approach lies in its capacity to continuously adapt the caching strategy to evolving network dynamics, thereby keeping data up to date. However, the increased computational overhead required for maintaining the freshness of transient data may affect the approach's feasibility, particularly in energy-constrained edge environments where reducing processing complexity is crucial.

Table 1 compares different ML techniques used for content caching in ICN. The table shows that DRL provides significant improvements in improving cache hit rate as compared to the traditional caching schemes. The use of any

specific ML model depends specifically in the requirements of IoT environment.

III. THE PROPOSED SCHEME

The integration of IoT with ICN requires an efficient cache management schemes. ELC scheme is improved using a content popularity-based decision process, which is designed to adapt to rapid changes in IoT networks. Using this approach the popular contents are cached near the user, which improves responsiveness and efficiency. Figure 2, shows the proposed EELC, which employs a reinforced learning algorithm known as PPO to improve adaptability and efficiency of caching in Information-Centric Networking (ICN) for IoT environments. The essential function of edge node is to connect IoT devices to the cache nodes and subscriber. Moreover, it also utilizes the PPO algorithm to make intelligent caching decisions as shown in 2. In addition, edge node also manages a path table, which includes information about all the content cached throughout the network. This path table optimizes network performance by directing content requests to the most appropriate cache node.

The use of PPO algorithm allows the edge node to learn frequently from the IoT network environment, which helps in tuning caching scheme accordingly. This ensures that the machine learning based scheme caches popular and frequently requested contents near users, thus reducing latency accessing information, and improves responsiveness of the Information stored on leaf nodes, which also act as access points for user. Therefore the system is designed to ensure that resources are brought near where they are required the most. This will reduce the number of access towards the IoT devices reducing its energy requirement.

In addition, the PPO algorithm enables the system to adapt to the dynamic network environment and user needs, providing a flexible and efficient means of content delivery. When user communicates with the network through using leaf node the PPO-powered edge node ensures fast and efficient delivery of content employing regularly updated path tables for making routing and caching decisions. The details of the caching decision process is provided in the subsequent section.

A. CACHING DECISION PROCESS

The EELC process, starts when a content request is made at the edge node. The caching decision is based on the popularity of the content, which is determined by the number of request or frequency for a content and current status of the network. Popular content takes are given precedence for caching, thus which helping optimize network resources.

1) OBSERVATION OF THE NETWORK STATE

Understanding of the current state of the network is the foundational step in determining of how to manage caching. This process involves measuring key factors such as how much a content is being requested and how much storage and computing power is available to make decisions regarding

TABLE 1. Summary of related works.

S.No.	Citation	Author(s)	ML Technique	Issue Addressed	Pros and Cons
1	[19]	He et al.	Deep Reinforcement Learning (DRL)	Integration of networking, caching, and computing in connected vehicles	Pros: Enhances delay-sensitive applications; Cons: Requires substantial computational resources
2	[17]	Zhu et al.	DRL	Caching IoT data at network edges	Pros: Improves over traditional strategies; Cons: High computational demand for model training
3	[21]	Wu et al.	DRL	Dynamic caching decisions based on data characteristics	Pros: Adaptable, improves cache management; Cons: Limited applicability in constrained environments
4	[22]	Herouala et al.	Data Classification	Aggregation in NDN for dynamic IoT	Pros: Improves network performance; Cons: Requires adaptation for dynamic features
5	[23]	Xu et al.	Deep Q-Learning (DQ-Learning)	Computing and caching resource management in IC-IoT	Pros: Maximizes rewards by caching popular AI models; Cons: Requires balancing immediate and long-term benefits
6	[24]	Narayanan et al.	LSTM Encoder-Decoder	Framework for content caching	Pros: Significantly improves cache hit rates; Cons: Integration complexity with existing policies
7	[25]	Hou et al.	Graph Neural Network (GNN)	Caching strategy in NDN	Pros: Improves cache hit ratios and reduces latency; Cons: Implementation challenges in scalable environments
8	[26]	Weerasinghe et al.	Reinforcement Learning (RL)	Intelligent cache resource management	Pros: Predicts context changes, enhances content availability; Cons: Dependent on precise context prediction
9	[27]	Hou et al.	GNN	Proactive caching decisions in NDN	Pros: Anticipates content popularity; Cons: Complexity and data requirements for GNN models
10	[30]	Kang et al.	Seq2seq LSTM	Forecasting content popularity for caching	Pros: Processes sequential data effectively; Cons: Demands detailed historical data, complexity
11	[31]	Gupta et al.	User Behavior Analysis	Individualized caching options in IoT	Pros: Improves cache hit rate; Cons: Privacy concerns with user data
12	[32]	Zhang et al.	Machine Learning (ML)	Intelligent caching in ICN-based IoT networks	Pros: Enhances user experience, optimizes resource use; Cons: Accuracy and timeliness of predictive data
13	[33]	Dhawan et al.	Deep Reinforced Learning	Transient content caching and replacement	Pros: Bolsters cache utilization and efficiency; Cons: Reliance on historical data
14	[34]	Sharma et al.	Markov Decision Process (MDP)	Optimal caching for data freshness	Pros: Ensures data timeliness, reduces energy; Cons: High computational resources required
15	[35]	Karras et al.	DRL with MCMC	Caching in IoT ecosystems	Pros: Balances data availability and energy efficiency; Cons: Computational complexity for real-time deployment
16	[36]	Li et al.	RL	Resource scheduling in heterogeneous edge computing networks	Pros: Elevates performance and reliability; Cons: Scalability and computational resource demands
17	[37]	Aung et al.	DRL	Content caching in vehicular social networks	Pros: Handles high mobility and variable network density; Cons: Requires frequent retraining
18	[29]	Papenfuß et al.	Hyperparameter Optimization	Energy efficiency in NDN for IoT	Pros: Achieves considerable energy savings; Cons: Complexity limits real-time applicability
19	[39]	Delvadia et al.	Q-learning	Optimized forwarding strategy for ICN	Pros: Reduces network congestion and improves performance; Cons: Slow adaptation to rapid changes
20	[38]	Wang et al.	DRL	Intelligent Edge Caching (ICE)	Pros: Improves cache utilization and resource management; Cons: Increased complexity, requires validation
21	[40]	Yan et al.	Actor-Critic DRL	Content fetching strategies for D2D networks	Pros: Minimizes collision, optimizes throughput; Cons: Increased complexity, requires high computational resources
22	[41]	Zhang et al.	TD-MEAC	Transient data caching	Pros: Enhances data freshness; Cons: High computational overhead

caching. Network state analysis is shown in Equation 1.

$$NS_t = \sum_{i=1}^N r_i(t) \times S_i(t) \quad (1)$$

This equation calculates the network state NS_t at time t by adding the product of the request rate $r_i(t)$ for each content i and available storage $S_i(t)$. A higher NS_t indicates better network readiness to handle content requests.

To further improve this observation, we consider the network load:

$$L_t = \sum_{i=1}^N \frac{r_i(t)}{B_i(t)} \quad (2)$$

Equation 2 calculates the network load L_t at time t by adding the ratio of the request rate $r_i(t)$ to the available bandwidth $B_i(t)$ for each content i . A lower L_t indicates a less congested network. Equation 2 is used to calculate the expected energy efficiency based on the caching probability, which helps in deciding the optimal caching location to maximize efficiency.

Combining the network state and load gives us an overall network efficiency metric:

$$E_t = \frac{NS_t}{L_t} \quad (3)$$

The network efficiency E_t at time t is determined by dividing the network state NS_t by the network load L_t .

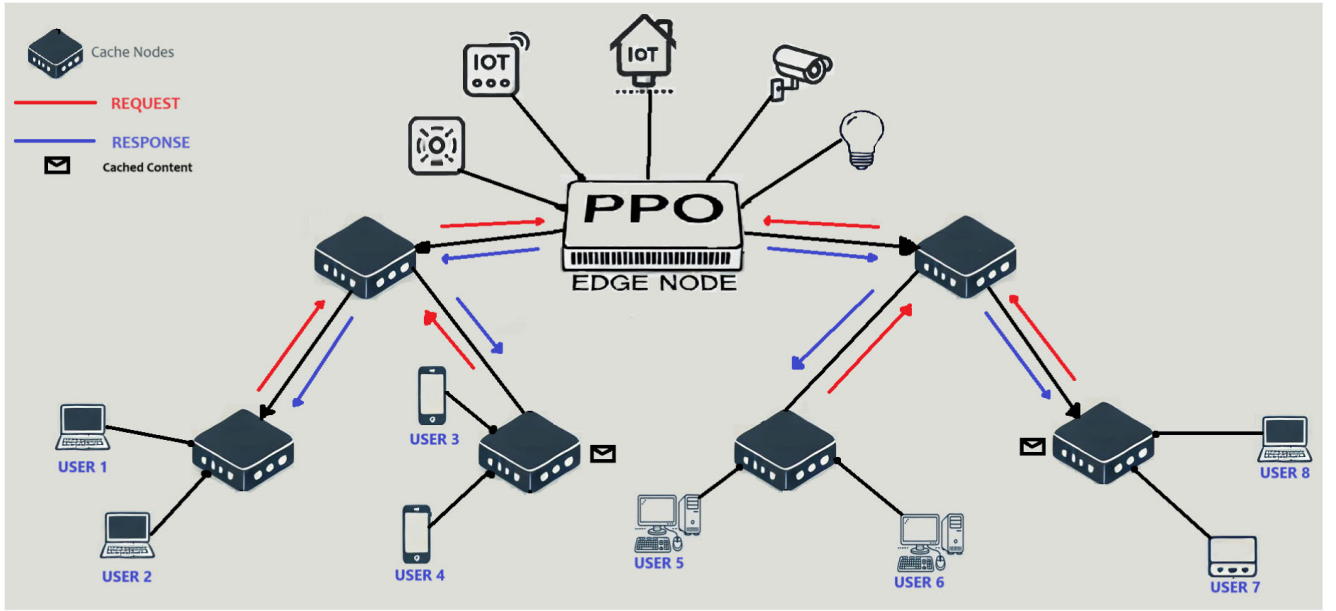


FIGURE 2. Enhanced edge-linked caching.

A higher E_t suggests that the network is operating efficiently with respect to both content demand and resource availability. Equation 3 show the reward function, which is used by reinforcement learning model. It is used for optimizing caching decision by balancing content freshness and energy consumption.

2) CONTENT POPULARITY-BASED CACHING DECISION

Once the network state is observed, the next step involves calculating the popularity of each content to make data-driven caching decisions.

$$P_i = \frac{\sum_{t=0}^T r_i(t)}{T} \quad (4)$$

The popularity P_i of content i is computed as the average request rate over a period T . This metric reflects how frequently a content is requested over time, which is essential for determining its caching priority.

To capture the temporal dynamics of content popularity, a decaying factor can be introduced:

$$P_i(t) = \sum_{k=0}^T r_i(t-k) \times \gamma^k \quad (5)$$

Here, $P_i(t)$ represents the time-dependent popularity of content i at time t , with γ being the decay factor ($0 < \gamma < 1$). This equation weights recent requests more heavily than older ones, dynamically adjusting content popularity over time.

3) IMPLEMENTING CACHING DECISIONS

With content popularity established, the system prioritizes caching decisions by balancing popularity with storage

constraints.

$$CP_i = \alpha P_i + (1 - \alpha) \frac{1}{S_i} \quad (6)$$

Caching priority CP_i is calculated by weighting the content popularity P_i against the inverse of its storage requirement S_i , where α is a balancing factor. This ensures that popular, low-storage content is given higher caching priority.

To account for varying network conditions, the caching decision can be further refined by incorporating the network efficiency metric:

$$CP_i(t) = \beta CP_i \times E_t \quad (7)$$

This equation adjusts the caching priority $CP_i(t)$ at time t by multiplying it with the network efficiency E_t , where β is a tuning parameter. This adjustment allows the system to optimize resource utilization based on real-time network conditions.

4) FEEDBACK AND ITERATIVE IMPROVEMENT

Continuous monitoring and iterative adjustments ensure that the caching strategy remains effective over time.

$$\Delta CP_i = \eta(\hat{R}_i - R_i) \quad (8)$$

The adjustment ΔCP_i in caching priority for content i is calculated using a learning rate η , based on the difference between the observed cache hit rate $\hat{R}_i$ and the expected rate R_i . This facilitates continuous improvement in the caching strategy.

To enhance the learning process, a momentum term can be incorporated:

$$\Delta CP_i(t+1) = \mu \Delta CP_i(t) + \eta(\hat{R}_i - R_i) \quad (9)$$

Algorithm 1 PPO-Based Caching Decision Algorithm

- 1: **Input:** Content request r_i
- 2: **Output:** Caching decision a_i
- 3: Initialize policy network π_θ with parameters θ , learning rate $\alpha = 0.00025$
- 4: Define reward function $R(a_i)$ (positive for cache hits, negative for misses and high energy use)
- 5: Set discount factor $\gamma = 0.99$
- 6: **Network architecture:** Multilayer perceptron (2 hidden layers, 64 neurons each, ReLU activation)
- 7: Set update frequency to 2000 timesteps
- 8: Use entropy regularization with coefficient $\beta = 0.01$ for exploration-exploitation balance
- 9: Observe initial network state S_0
- 10: **for** each content request r_i **do**
- 11: Observe current state S_i
- 12: Compute action probabilities $P(a_i|S_i) = \pi_\theta(S_i)$
- 13: **if** $P(\text{Cache}) > P(\text{Direct})$ **then**
- 14: Set action $a_i = \text{Cache}$
- 15: Execute caching action a_i
- 16: **else**
- 17: Set action $a_i = \text{Direct delivery}$
- 18: Execute direct delivery action a_i
- 19: Observe new state S_{i+1}
- 20: Evaluate reward R_i based on a_i
- 21: Store transition (S_i, a_i, R_i, S_{i+1}) in memory buffer $\mathcal{D}$
- 22: Update policy network π_θ using PPO and stored transitions from $\mathcal{D}$

Here, μ represents the momentum factor, which helps smooth the adjustments over time, ensuring that sudden changes in the network do not cause drastic alterations in the caching strategy.

5) ADAPTATION TO NETWORK CHANGES

The system adapts to network dynamics by continuously updating the caching strategy based on the feedback mechanism.

$$A_i(t+1) = A_i(t) + \Delta CP_i \quad (10)$$

The adapted caching strategy $A_i(t+1)$ at time $t+1$ is updated by adding the adjustment ΔCP_i to the strategy from the previous time stamp. This ensures that the caching strategy evolves with the changing network conditions.

Additionally, to prevent overfitting to specific patterns, a regularization term can be introduced:

$$A_i(t+1) = A_i(t) + \Delta CP_i - \lambda \|A_i(t)\| \quad (11)$$

In this equation, λ is the regularization parameter, and $\|A_i(t)\|$ is the norm of the caching scheme. This regularization term penalizes overly complex schemes, promoting generalization and robustness in the caching decisions.

The PPO-Based Caching Decision Algorithm (Algorithm 1), which starts by initializing the policy network

π_θ with parameters θ and a learning rate $\alpha = 0.00025$, chosen for stable and gradual updates, avoiding large jumps in parameter values. The reward function $R(a_i)$ gives positive rewards for cache hits and penalizes cache misses or high energy use. The discount factor $\gamma = 0.99$ emphasizes immediate rewards while still considering long-term gains. The network architecture has two hidden layers of 64 neurons with ReLU activation and updates every 2000 timesteps, balancing efficient learning and computational load. Entropy regularization with $\beta = 0.01$ helps maintain exploration and avoid premature convergence. For each content request r_i , the current state S_i is observed, and action probabilities $P(a_i|S_i)$ are computed. If caching is favored, action a_i is set to “Cache” and executed; otherwise, direct delivery is chosen. After execution, the new state S_{i+1} is observed, and the reward R_i is evaluated based on the action. The transition (S_i, a_i, R_i, S_{i+1}) is stored in memory buffer $\mathcal{D}$, and the policy network is updated using PPO from stored transitions. This loop repeats for each request, ensuring continuous policy improvement as shown in Figure 3.

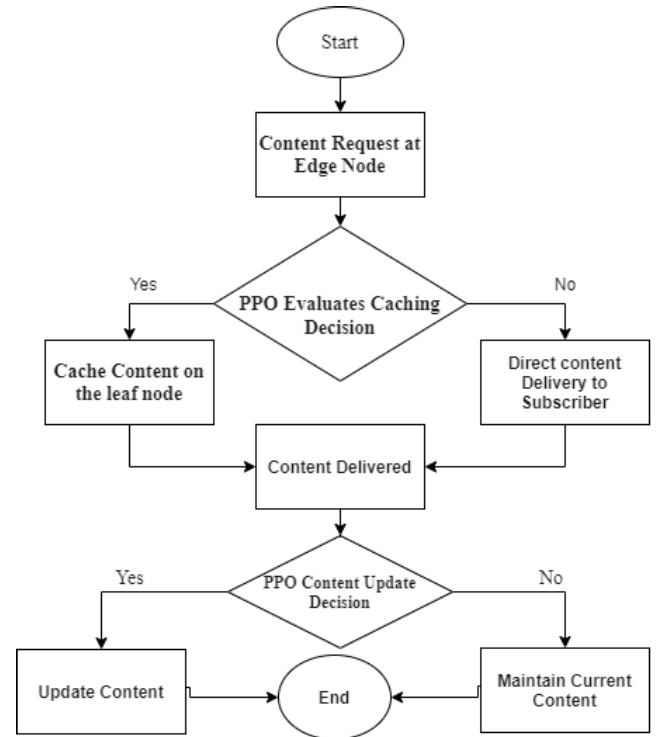


FIGURE 3. Flowchart for PPO-based caching decision.

B. CONTENT UPDATE IN ENHANCED ELC

The process of determining content updates in the EELC system is driven by content popularity metrics. This approach ensures that content frequently requested by users and important to the network is kept up-to-date. The decision to update content is based on both its current demand and its relevance within the network, optimizing overall network performance.

1) OBSERVING THE NETWORK STATE FOR CONTENT UPDATE NEEDS

The first step in the content update process is a comprehensive assessment of the current network state, focusing specifically on two critical factors: the age of the content and its popularity. The edge node systematically analyzes how long each piece of content has been in the system and how frequently it is requested by users.

a: CONTENT AGE CALCULATION

$$A_i(t) = t - T_i \quad (12)$$

Here, $A_i(t)$ represents the age of content i at time t , with T_i as the last time the content was updated or cached.

To normalize the age and reflect its significance in decision-making, the age can be weighted using a logistic function:

$$\tilde{A}_i(t) = \frac{1}{1 + e^{-\kappa(A_i(t) - A_0)}} \quad (13)$$

where $\tilde{A}_i(t)$ is the normalized age of content i , κ controls the steepness of the curve, and A_0 is a reference age above which content is increasingly considered outdated.

b: CONTENT POPULARITY CALCULATION

$$P_i(t) = \sum_{k=0}^T r_i(t-k) \times \gamma^k \quad (14)$$

$P_i(t)$ is the time-dependent popularity of content i at time t , where $r_i(t-k)$ represents the request rate at time $t-k$, and γ (with $0 < \gamma < 1$) is a decay factor that gives more weight to recent requests.

c: COMPOSITE IMPORTANCE SCORE

$$I_i(t) = \alpha \cdot \tilde{P}_i(t) - \beta \cdot \tilde{A}_i(t) \quad (15)$$

This equation calculates the composite importance score $I_i(t)$ for content i at time t , where $\tilde{P}_i(t)$ is the normalized popularity of content i and $\tilde{A}_i(t)$ is the normalized age. The parameters α and β are weights that balance the influence of popularity and age on the content update decision.

2) POLICY NETWORK FOR DECISION MAKING

Traditionally, content update decisions might involve a PPO policy network. However, in this revised approach, a heuristic method is employed. This heuristic method applies a set of predefined rules or formulas to evaluate whether updating certain content, based on its current popularity, will contribute positively to network performance.

a: HEURISTIC FUNCTION

$$H_i(t) = \lambda_1 \cdot I_i(t) + \lambda_2 \cdot \frac{1}{B_i(t)} + \lambda_3 \cdot E_t \quad (16)$$

The heuristic score $H_i(t)$ for content i at time t is determined by combining the composite importance score $I_i(t)$, the

inverse of the available bandwidth $B_i(t)$, and the network efficiency E_t . The parameters λ_1 , λ_2 , and λ_3 are weights that control the influence of each factor on the heuristic score.

3) MAKING CONTENT UPDATE DECISIONS

Based on the information gathered, the system makes decisions regarding which content needs to be updated. These decisions hinge on the content's current demand and relevance. If a piece of content is highly requested and considered important to the user base, it is marked for an update to ensure it remains current and useful.

a: CONTENT UPDATE DECISION RULE

$$\text{Update Content } i \text{ if } H_i(t) > \theta \quad (17)$$

Content i should be updated if its heuristic score $H_i(t)$ exceeds a predefined threshold θ . This threshold helps in filtering out content that does not require immediate updates.

b: THRESHOLD UPDATE RULE

$$\theta(t+1) = \theta(t) + \eta \cdot \Delta H_i(t) \quad (18)$$

The threshold θ is dynamically adjusted based on observed network performance, with η as the learning rate, and $\Delta H_i(t)$ as the difference between the current heuristic score and a target score H_{target} .

4) REWARD MECHANISM AND POLICY OPTIMIZATION

To optimize the content update strategy, a reward mechanism is implemented. This mechanism aligns with the popularity-based approach for updating content, ensuring that the most impactful and beneficial updates are prioritized.

a: REWARD FUNCTION

$$R_i(t) = \lambda_1 \cdot \Delta P_i(t) - \lambda_2 \cdot \Delta L_i(t) + \lambda_3 \cdot \Delta S_i(t) \quad (19)$$

The reward $R_i(t)$ for updating content i at time t is defined by combining improvements in popularity $\Delta P_i(t)$, latency reduction $\Delta L_i(t)$, and storage efficiency gain $\Delta S_i(t)$. The parameters λ_1 , λ_2 , and λ_3 are weights that balance the different reward components.

5) CONTINUOUS ADAPTATION AND IMPROVEMENT

The content update strategy is designed to be flexible and responsive to evolving content demands and network conditions. As user preferences and network dynamics evolve, the strategy adjusts to ensure the network remains effective and efficient in its content delivery.

a: WEIGHT UPDATE RULE

$$\lambda_i(t+1) = \lambda_i(t) + \eta \cdot \frac{\partial R_i(t)}{\partial \lambda_i} \quad (20)$$

The weights λ_i used in the reward function are continuously updated based on the gradient of the reward function with respect to each weight, ensuring that the content update strategy evolves to maintain optimal network performance.

b: OPERATIONAL SCENARIO OF CONTENT UPDATE DETERMINATION IN EELC

In this process, content update decisions are centralized at the edge node, driven by popularity metrics. The edge node observes network conditions, assessing content age and popularity, to determine the necessity of content updates. The decision-making process is augmented by a heuristic method that evaluates the potential impact of updating content based on its current demand and relevance. The role of the leaf nodes is to signal content access patterns to the edge node, which, along with the path table data, informs the decision-making process. The policy network, driven by these insights, makes calculated decisions on content updates. The effectiveness of these decisions is measured through a reward mechanism, which evaluates improved accessibility and reduced latency for popular content. Continuous adaptation is a cornerstone of this strategy, allowing for real-time adjustments to align with evolving content demands and network dynamics.

6) PPO CONFIGURATION FOR CONTENT UPDATE DETERMINATION

In the EELC system, the PPO configuration for content update determination is strategically tailored. The learning rate is set at $\alpha = 0.001$, higher than the caching decision process, enabling quicker adaptation to changing user preferences. The reward function is designed to incentivize updating high-demand content, with positive rewards assigned for frequently accessed updates:

a: PPO REWARD FUNCTION

$$R_i(t) = \alpha_1 \cdot \text{Update Success} - \alpha_2 \cdot \text{Latency Penalty} \quad (21)$$

The reward function used in the PPO policy network balances the success of content updates against any latency penalties incurred, with parameters α_1 and α_2 controlling this balance.

The discount factor $\gamma = 0.95$ prioritizes immediate over long-term outcomes, crucial in the fast-evolving content landscape. The policy network architecture includes one or two hidden layers with 128 neurons each, capable of capturing complex decision-making dynamics. Update frequency is set to every 1000 timesteps, ensuring rapid policy adaptation. Finally, an entropy coefficient of 0.02 maintains a balance between exploration and exploitation:

b: PPO LOSS FUNCTION

$$L(\theta) = - \sum_t \log \pi_\theta(a_t|s_t) \cdot R_t + \beta \cdot H(\pi_\theta) \quad (22)$$

The loss function for the PPO policy network includes the log probability of actions, cumulative rewards, and an entropy term to balance exploration and exploitation. The parameter β controls the influence of entropy in the loss function.

This configuration allows the PPO to effectively adapt to new patterns in content popularity and network traffic, ensuring that the EELC system remains robust and efficient.

Algorithm 2 PPO-Based Content Update Determination Algorithm

```

1: Input: Content update check  $u$ 
2: Output: Update decision  $a_u$ 
3: Initialize policy network  $\pi_\theta$ , parameters  $\theta$ 
4: Set  $\alpha = 0.001$ ,  $\gamma = 0.95$ 
5: Reward function  $R(a_u)$ : positive for high-demand updates
6: Network: 1-2 hidden layers, 128 neurons, ReLU activation
7: Update every 1000 steps, entropy regularization  $\beta = 0.02$ 
8: Observe initial state  $S_0$ 
9: for each update check  $u$  do
10:   Observe state  $S_u$  (age, frequency)
11:   Pre-process  $S_u$  (e.g., normalize age, scale frequency)
12:   Compute  $P(a_u|S_u) = \pi_\theta(S_u)$ 
13:   if  $P(\text{Update}) > P(\text{No Update})$  then
14:     Set  $a_u = \text{Update}$ 
15:     Execute update
16:   else
17:     Set  $a_u = \text{No Update}$ 
18:     Skip update
19:   Observe new state  $S_{u+1}$ 
20:   Compute reward  $R_u$  for  $a_u$ 
21:   Store  $(S_u, a_u, R_u, S_{u+1})$  in buffer  $\mathcal{D}$ 
22: Update  $\pi_\theta$  using PPO and transitions from  $\mathcal{D}$ 

```

In the PPO-Based content update determination algorithm (Algorithm 2), which starts with the initialization the policy network π_θ with parameters θ in step 3, followed by setting the learning rate $\alpha = 0.001$ in step 4 to ensure stable and gradual updates. In step 5, the discount factor $\gamma = 0.95$ is set, prioritizing near-term rewards while considering future outcomes. The reward function $R(a_u)$ in step 6 assigns positive rewards for high-demand content updates. Step 7 defines the network architecture with 1-2 hidden layers and 128 neurons, using ReLU activation for non-linearity. Step 8 sets the update frequency to every 1000 steps, and step 9 uses entropy regularization with $\beta = 0.02$ to balance exploration and exploitation. Step 10 observes the initial state S_0 , and in step 11, the current state S_u (content age and request frequency) is observed. Step 12 pre-processes S_u , normalizing and scaling input features. Step 13 computes action probabilities $P(a_u|S_u)$ via the policy network. Steps 14-16 make decisions: if $P(\text{Update}) > P(\text{No Update})$, action $a_u = \text{Update}$ is taken and executed; otherwise, the update is skipped. Step 17 observes the new state S_{u+1} , while step 18 computes the reward R_u based on the action taken. Step 19 stores the transition (S_u, a_u, R_u, S_{u+1}) in the memory buffer $\mathcal{D}$, and step 20 updates the policy network π_θ using the PPO algorithm with the stored transitions. The parameter values $\alpha = 0.001$, $\gamma = 0.95$, and $\beta = 0.02$ are chosen to maintain stable learning, emphasize near-term

gains, and ensure proper exploration. The update frequency of 1000 steps balances model adaptability and computational efficiency. The flowchart for content update is shown in Figure 4.

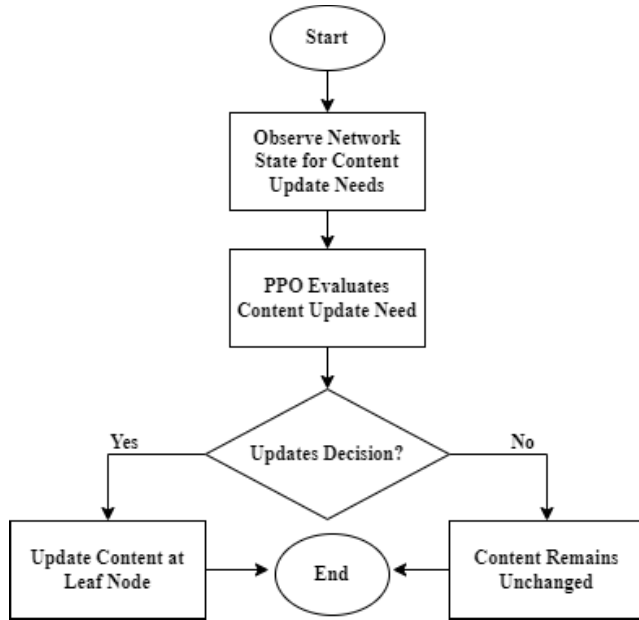


FIGURE 4. Flowchart for content update in EELC.

IV. SIMULATION ENVIRONMENT

The simulation was carried out in a virtual network environment spanning 500×500 meters, encompassing 1,000 nodes with diverse functionalities. The node distribution included 10 publishers employing NB-IoT technology, 5 edge nodes, 15 leaf nodes, 45 intermediate nodes, and 975 subscribers utilizing the IEEE802.11a WiFi standard. A Zipf distribution, with α ranging from 0.3 to 1.1, was utilized to model the content request patterns. Scenarios with cache capacities of 10 GB and 32 GB were simulated, each across 10 iterations to ensure robustness of results. Performance benchmarks were established by comparing the proposed Enhanced EELC scheme against RL-based Scheme TD-MEAC and the traditional ELC and LCE schemes, focusing on energy consumption, cache hit ratio, and server hit reduction ratio.

V. RESULTS AND ANALYSIS

A. ENERGY CONSUMPTION

The energy consumption at cache size 10 GB and 32 GB is shown in Figure 5 and 6 respectively. Moreover, Table 2 shows the average consumption values at both sizes with respect to different values of α . The EELC strategy exhibited the lowest energy consumption per request, particularly as the α value approached 1.1. This efficiency is primarily due to the PPO algorithm's role in the edge nodes, which intelligently decides on caching popular content, thus diminishing the energy-intensive data transmissions from the IoT publisher. Similarly, when the cache has been increased to 32 GB, the

EELC maintained its lead in energy efficiency. This trend shows the proposed scheme is effective in utilizing larger storage capacities to reduce energy usage across the network.

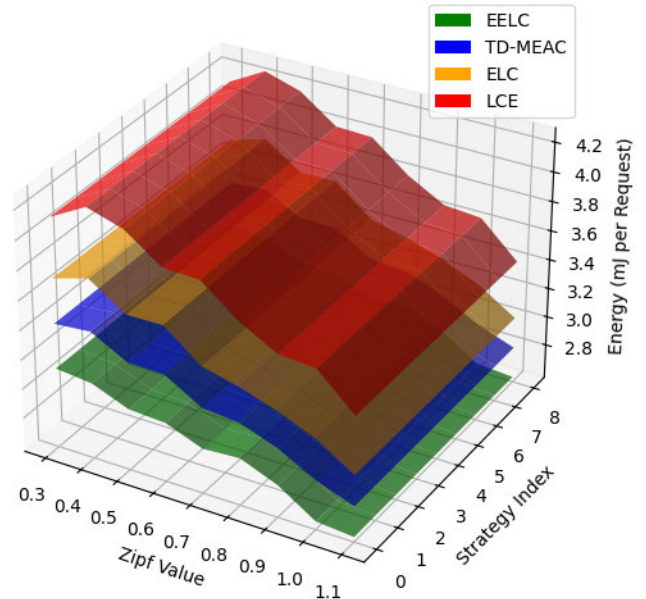


FIGURE 5. Energy consumption comparison for 10 GB cache.

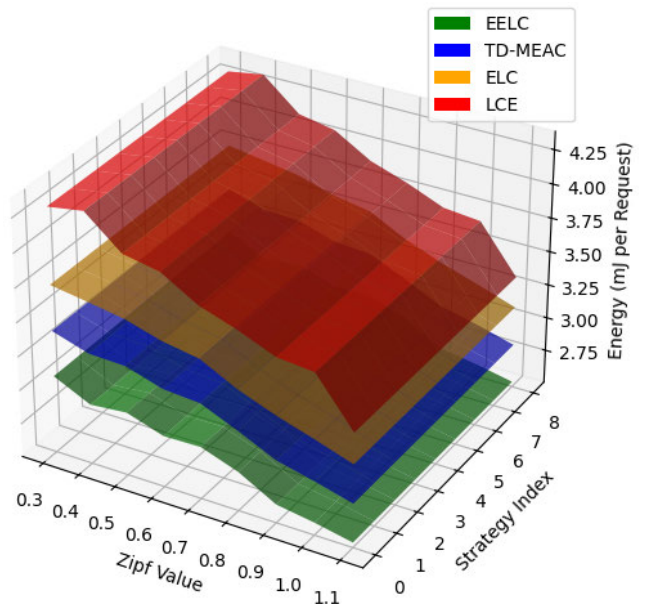


FIGURE 6. Energy consumption comparison for 32 GB cache.

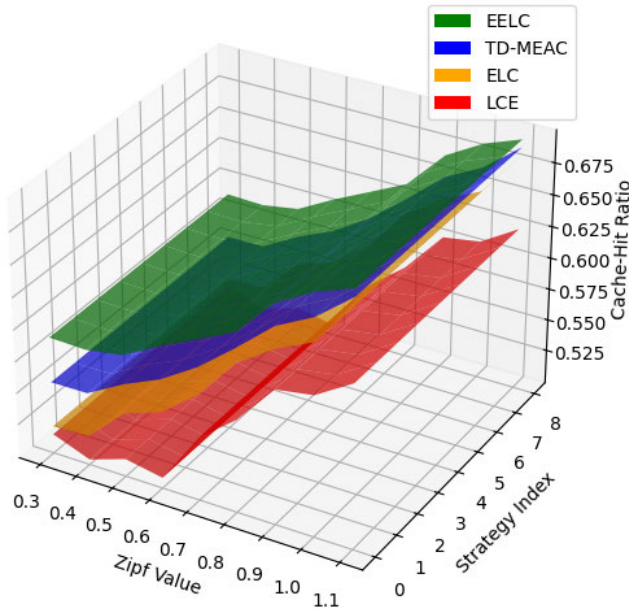
B. CACHE HIT RATIO

Figure 7 and Figure 8 shows the cache hit ratio at cache size 10 GB and 32 GB. Table 3 shows the cache hit ratio at different values of α . In the scenario with a 10 GB cache, EELC reported superior cache hit ratios over TD-MEAC, ELC, and LCE. This outcome can be attributed to

TABLE 2. Energy consumption for 10 GB and 32 GB cache.

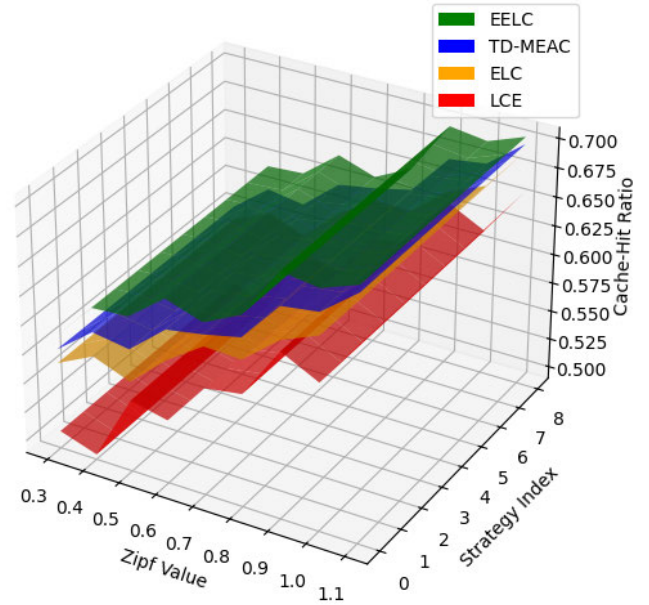
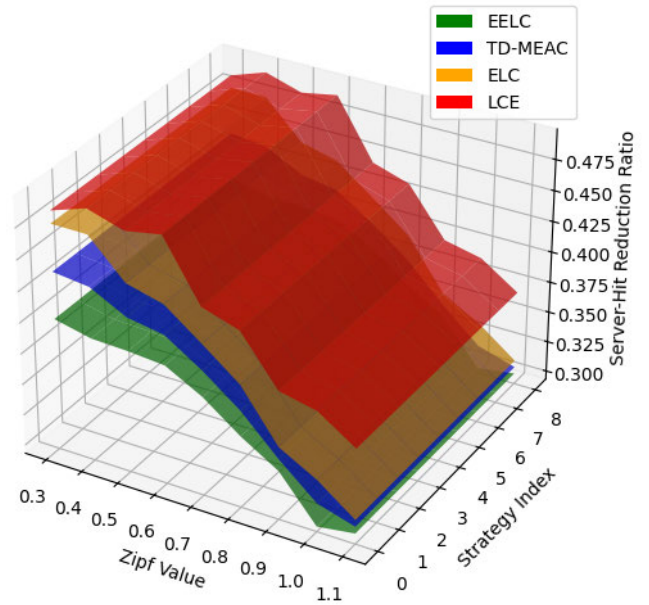
α	EELC	TD-MEAC	ELC	LCE
10 GB Cache				
0.3	3.112	3.418	3.723	4.128
0.4	3.085	3.439	3.793	4.261
0.5	2.987	3.307	3.628	4.187
0.6	2.972	3.301	3.631	4.010
0.7	2.876	3.155	3.434	4.001
0.8	2.891	3.132	3.372	3.806
0.9	2.789	3.039	3.290	3.668
1.0	2.625	2.904	3.184	3.649
1.1	2.602	2.812	3.022	3.416
32 GB Cache				
0.3	3.073	3.408	3.743	4.301
0.4	2.945	3.323	3.702	4.345
0.5	2.970	3.314	3.658	4.121
0.6	2.882	3.231	3.579	4.070
0.7	2.888	3.211	3.534	3.912
0.8	2.791	3.093	3.396	3.822
0.9	2.639	2.966	3.293	3.710
1.0	2.596	2.901	3.206	3.685
1.1	2.540	2.827	3.114	3.341

the PPO algorithm's predictive accuracy in content popularity, ensuring high-demand content availability within the proximity of user access points. The larger cache size further augmented the cache hit ratio for the EELC, confirming the PPO algorithm's capability to leverage additional storage space for accommodating popular content.

**FIGURE 7.** Cache hit ratio comparison for 10 GB cache.

C. SERVER HIT REDUCTION RATIO

The server hit reduction ratio is shown in Figure 9 and 10. Moreover, Table 4 shows the server hit reduction ratio at cache sizes 10 GB and 32 GB at different values of α . EELC significantly curtailed server requests, implying

**FIGURE 8.** Cache hit ratio comparison for 32 GB cache.**FIGURE 9.** Server hit reduction ratio comparison for 10 GB cache.

a more sustainable utilization of server resources and an optimized network operation compared to the other caching schemes. Mirroring the 10 GB cache results, the server hit reduction ratio for EELC continued to outshine, showcasing the strategy's robustness in larger cache environments.

D. COMPARATIVE PERFORMANCE ANALYSIS OF ENHANCED EDGE-LINKED CACHING

The EELC scheme exhibits better performance compared to TD-MEAC, ELC, and LCE in terms of energy consumption, cache hit ratio, and server hit reduction. This reason for

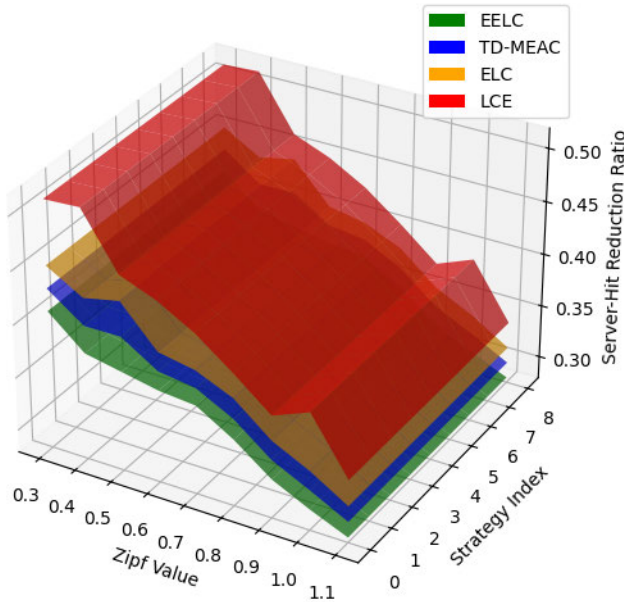


FIGURE 10. Server hit reduction ratio comparison for 32 GB cache.

TABLE 3. Cache hit ratio for 10 GB and 32 GB cache.

α	EELC	TD-MEAC	ELC	LCE
10 GB Cache				
0.3	0.591	0.555	0.520	0.513
0.4	0.590	0.556	0.521	0.502
0.5	0.595	0.572	0.549	0.512
0.6	0.610	0.585	0.559	0.507
0.7	0.624	0.603	0.582	0.548
0.8	0.638	0.627	0.615	0.571
0.9	0.669	0.652	0.635	0.604
1.0	0.686	0.667	0.648	0.609
1.1	0.696	0.690	0.684	0.627
32 GB Cache				
0.3	0.588	0.576	0.564	0.504
0.4	0.621	0.600	0.580	0.494
0.5	0.623	0.595	0.568	0.551
0.6	0.648	0.622	0.596	0.544
0.7	0.637	0.627	0.617	0.582
0.8	0.653	0.634	0.616	0.587
0.9	0.697	0.669	0.641	0.632
1.0	0.693	0.674	0.655	0.616
1.1	0.705	0.699	0.693	0.657

better performance is use of PPO algorithm. The Use of this algorithm allows EELC to constantly learn from the network and improve caching decisions thus maximizing caching hits, reduce server hits, and reduce power consumption unlike ELC and LCE, which relies on static heuristics that may not adapt well to network conditions.

TD-MEAC, which is based on the maximum entropy actor-critic algorithm, outperforms both ELC and LCE due to its capacity for learning and adaptation. By utilizing entropy to encourage exploration, TD-MEAC achieves better cache-hit ratios and server-hit reduction compared to the static strategies of ELC and the indiscriminate replication of LCE. However, the focus on extensive exploration in TD-MEAC also leads to frequent cache updates, resulting in

TABLE 4. Server hit ratio for 10 GB and 32 GB cache.

α	EELC	TD-MEAC	ELC	LCE
10 GB Cache				
0.3	0.400	0.437	0.475	0.485
0.4	0.393	0.434	0.475	0.495
0.5	0.392	0.419	0.446	0.486
0.6	0.389	0.414	0.438	0.492
0.7	0.373	0.394	0.416	0.444
0.8	0.351	0.372	0.392	0.435
0.9	0.333	0.342	0.352	0.397
1.0	0.297	0.316	0.335	0.390
1.1	0.302	0.307	0.313	0.370
32 GB Cache				
0.3	0.409	0.430	0.452	0.512
0.4	0.380	0.406	0.432	0.514
0.5	0.373	0.406	0.439	0.463
0.6	0.366	0.384	0.402	0.449
0.7	0.363	0.379	0.396	0.430
0.8	0.342	0.361	0.381	0.404
0.9	0.316	0.334	0.352	0.375
1.0	0.299	0.317	0.335	0.389
1.1	0.284	0.299	0.313	0.338

higher energy consumption. This approach, while effective at reducing server hits and increasing cache hits, does not balance exploration and exploitation as well as EELC, making it less energy efficient.

The EELC, by leveraging PPO, strikes an optimal balance between exploration and exploitation. It intelligently prioritizes caching popular content at the edge, thereby enhancing the cache-hit ratio, reducing server load, and significantly lowering energy consumption. This targeted approach not only reduces latency but also avoids the inefficiencies seen in LCE's redundant caching and the static limitations of ELC. Thus, EELC's real-time adaptability and efficient resource management make it the most effective strategy, outperforming TD-MEAC, ELC, and LCE across all key performance metrics.

VI. CONCLUSION

The study demonstrates the efficiency of the EELC algorithm, which incorporates Proximal Policy Optimization (PPO) for dynamic and efficient data processing in ICN-based IoT systems. This approach increases energy efficiency, cache hit rate, and reduces server load as compared to transient data caching strategy using a maximum entropy Actor-Critic (TD-MEAC), Edge-Linked Caching (ELC), and Leave Copy Everywhere (LCE) methods. In particular, EELC achieves increased energy efficiency up to 23.1% and a high cache hit ratio of 12.5%, as well as Reduces server hits up to 17.9 % compared to ELC in 10 GB cache environment. In 32 GB cache environment, EELC provides power up to 23.9%, a higher cache hit ratio of 10.3%, and PPO reduces server hits by 21.3% compared to ELC. PPO's integration allows for more nuanced decision making, using machine learning optimizing caching in a real-time IoT environment. Overall, the EELC system proves to be a robust solution for caching and updating, with provides nuanced network performance IoT environments.

REFERENCES

- [1] I. H. Sarker, A. I. Khan, Y. B. Abushark, and F. Alsolami, "Internet of Things (IoT) security intelligence: A comprehensive overview, machine learning solutions and research directions," *Mobile Netw. Appl.*, vol. 28, no. 1, pp. 296–312, 2023.
- [2] O. A. Wahab, A. Mourad, H. Otok, and T. Taleb, "Federated machine learning: Survey, multi-level classification, desirable criteria and future directions in communication and networking systems," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 2, pp. 1342–1397, 2nd Quart., 2021.
- [3] Z. Zhang, H. Ma, and L. Liu, "Cache-aware named-data forwarding in Internet of Things," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2015, pp. 1–6.
- [4] V. Jacobson, D. K. Smetters, J. D. Thornton, M. F. Plass, N. H. Briggs, and R. L. Braynard, "Network named content," in *Proc. 5th Int. Conf. Emerg. Netw. Exp. Technol.*, Dec. 2009, pp. 1–12.
- [5] I. U. Din, B.-S. Kim, S. Hassan, M. Guizani, M. Atiquzzaman, and J. J. P. C. Rodrigues, "Information-centric network-based vehicular communications: Overview and research opportunities," *Sensors*, vol. 18, no. 11, p. 3957, Nov. 2018.
- [6] D. Gupta, S. Rani, S. H. Ahmed, and R. Hussain, "Caching policies in NDN-IoT architecture," in *Integration of WSN and IoT for Smart Cities*. Cham, Switzerland: Springer, 2020, pp. 43–64.
- [7] Z. Zhang, C.-H. Lung, X. Wei, M. Chen, S. Chatterjee, and Z. Zhang, "In-network caching for ICN-based IoT (ICN-IoT): A comprehensive survey," *IEEE Internet Things J.*, vol. 10, no. 16, pp. 14595–14620, Aug. 2023.
- [8] N. Askar, A. Habbal, F. Z. Alden, X. Wei, H. Alaidaros, J. Guo, and H. Yu, "Forwarding strategies for named data networking based IoT: Requirements, taxonomy, and open research challenges," *IEEE Access*, vol. 11, pp. 78363–78383, 2023.
- [9] A. T. Herouala, B. Ziani, C. A. Kerrache, C. T. Calafate, N. Lagraa, and J.-C. Cano, "NBCC: Simulation of a new caching strategy using naive Bayes classifier in NDN," in *Proc. IEEE/ACM 26th Int. Symp. Distrib. Simul. Real Time Appl. (DS-RT)*, Sep. 2022, pp. 1–8.
- [10] K. Arulkumar, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Process. Mag.*, vol. 34, no. 6, pp. 26–38, Nov. 2017.
- [11] H. Asmat, I. U. Din, F. Ullah, M. Talha, M. Khan, and M. Guizani, "ELC: Edge linked caching for content updating in information-centric Internet of Things," *Comput. Commun.*, vol. 156, pp. 174–182, Apr. 2020.
- [12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [13] A. Rahman, K. Hasan, D. Kundu, M. J. Islam, T. Debnath, S. S. Band, and N. Kumar, "On the ICN-IoT with federated learning integration of communication: Concepts, security-privacy issues, applications, and future perspectives," *Future Gener. Comput. Syst.*, vol. 138, pp. 61–88, Jan. 2023.
- [14] K. Hasan and S.-H. Jeong, "An LSTM-based mobility prediction mechanism in the ICN-based vehicular networks," in *Proc. 27th Asia-Pacific Conf. Commun. (APCC)*, Oct. 2022, pp. 244–246.
- [15] L. Zhao, Y. Ran, H. Wang, J. Wang, and J. Luo, "Towards cooperative caching for vehicular networks with multi-level federated reinforcement learning," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2021, pp. 1–6.
- [16] T.-V. Nguyen, N.-N. Dao, V. Dat Tuong, W. Noh, and S. Cho, "User-aware and flexible proactive caching using LSTM and ensemble learning in IoT-MEC networks," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3251–3269, Mar. 2022.
- [17] H. Zhu, Y. Cao, X. Wei, W. Wang, T. Jiang, and S. Jin, "Caching transient data for Internet of Things: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2074–2083, Apr. 2019.
- [18] M.-O. Pahl, S. Liebold, and L. Wüstrich, "Machine-learning based IoT data caching," in *Proc. IFIP/IEEE Symp. Integr. Netw. Service Manage. (IM)*, Apr. 2019, pp. 9–12.
- [19] Y. He, N. Zhao, and H. Yin, "Integrated networking, caching, and computing for connected vehicles: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 67, no. 1, pp. 44–55, Jan. 2018.
- [20] Z. Zhang, C.-H. Lung, M. St-Hilaire, and I. Lambadaris, "Smart proactive caching: Empower the video delivery for autonomous vehicles in ICN-based networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7955–7965, Jul. 2020.
- [21] H. Wu, A. Nasehzadeh, and P. Wang, "A deep reinforcement learning-based caching strategy for IoT networks with transient data," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13310–13319, Dec. 2022.
- [22] A. T. Herouala, B. Ziani, C. A. Kerrache, A. E. Karim Tahari, N. Lagraa, and S. Mastorakis, "CaDaCa: A new caching strategy in NDN using data categorization," *Multimedia Syst.*, vol. 29, no. 5, pp. 2935–2950, Oct. 2023.
- [23] F. Xu, F. Yang, S. Bao, and C. Zhao, "DQN inspired joint computing and caching resource allocation approach for software defined information-centric Internet of Things network," *IEEE Access*, vol. 7, pp. 61987–61996, 2019.
- [24] A. Narayanan, S. Verma, E. Ramadan, P. Babaie, and Z.-L. Zhang, "DeepCache: A deep learning based framework for content caching," in *Proc. Workshop Netw. Meets AI & ML*, 2018, pp. 48–53.
- [25] J. Hou, H. Xia, H. Lu, and A. Nayak, "A graph neural network approach for caching performance optimization in NDN networks," *IEEE Access*, vol. 10, pp. 112657–112668, 2022.
- [26] S. Weerasinghe, A. Zaslavsky, S. W. Loke, A. Hassani, A. Medvedev, and A. Abken, "Adaptive context caching for IoT-based applications: A reinforcement learning approach," *Sensors*, vol. 23, no. 10, p. 4767, May 2023.
- [27] J. Hou, H. Lu, and A. Nayak, "A GNN-based proactive caching strategy in NDN networks," *Peer-Peer Netw. Appl.*, vol. 16, no. 2, pp. 997–1009, Mar. 2023.
- [28] J. Hou, T. Tao, H. Lu, and A. Nayak, "Intelligent caching with graph neural network-based deep reinforcement learning on SDN-based ICN," *Future Internet*, vol. 15, no. 8, p. 251, Jul. 2023.
- [29] D. Papenfuß, B. Gerlach, S. Fischer, and M. A. Hail, "Enhancing energy efficiency in IoT-NDN via parameter optimization," *Future Internet*, vol. 16, no. 2, p. 61, Feb. 2024. [Online]. Available: <https://www.mdpi.com/1999-5903/16/2/61>
- [30] M. W. Kang and Y. W. Chung, "Content caching based on popularity and priority of content using seq2seq LSTM in ICN," *IEEE Access*, vol. 11, pp. 16831–16842, 2023.
- [31] D. Gupta, S. Rani, S. H. Ahmed, S. Verma, M. F. Ijaz, and J. Shafi, "Edge caching based on collaborative filtering for heterogeneous ICN-IoT applications," *Sensors*, vol. 21, no. 16, p. 5491, Aug. 2021.
- [32] Z. Zhang, X. Wei, C.-H. Lung, and Y. Zhao, "ICache: An intelligent caching scheme for dynamic network environments in ICN-based IoT networks," *IEEE Internet Things J.*, vol. 10, no. 2, pp. 1787–1799, Jan. 2023.
- [33] G. Dhawan, A. P. Mazumdar, and Y. K. Meena, "PoSiF: A transient content caching and replacement scheme for ICN-IoT," *Wireless Commun. Mobile Comput.*, vol. 2023, pp. 1–18, Nov. 2023.
- [34] S. Sharma, A. Chaudhary, and S. K. Peddoju, "Optimal caching strategy for data freshness in IoT applications with edge computing," in *Proc. IEEE 49th Conf. Local Comput. Netw. (LCN)*, Oct. 2024, pp. 1–8.
- [35] A. Karras, C. Karras, I. Karydis, M. Avlonitis, and S. Sioutas, "An adaptive, energy-efficient drl-based and mcmc-based caching strategy for IoT systems," in *Proc. Int. Symp. Algorithmic Aspects Cloud Comput.* Cham, Switzerland: Springer, 2023, pp. 66–85.
- [36] Y. Li and B. Wu, "Software-defined heterogeneous edge computing network resource scheduling based on reinforcement learning," *Appl. Sci.*, vol. 13, no. 1, p. 426, Dec. 2022.
- [37] N. Aung, S. Dhelim, L. Chen, W. Zhang, A. Lakas, and H. Ning, "VeSoNet: Traffic-aware content caching for vehicular social networks based on path planning and deep reinforcement learning," 2021, *arXiv:2111.05567*.
- [38] T. Wang, Y. Deng, J. Mao, M. Chen, G. Liu, J. Di, and K. Li, "Towards intelligent adaptive edge caching using deep reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9289–9303, Oct. 2024.
- [39] K. Delvadia and N. Dutta, "Reinforcement learning inspired forwarding strategy for information centric networks using Q-learning algorithm," *Int. J. Commun. Syst.*, vol. 37, no. 6, p. e5707, Apr. 2024, doi: 10.1002/dac.5707.
- [40] M. Yan, M. Luo, C. A. Chan, A. F. Gyax, and C. Li, "Energy-efficient content fetching strategies in cache-enabled D2D networks via an actor-critic reinforcement learning structure," *IEEE Trans. Veh. Technol.*, pp. 1–11, Jun. 2024.
- [41] Y. Zhang, N. Chen, S. Yu, and L. Hu, "Transient data caching based on maximum entropy actor-critic in Internet-of-Things networks," *Int. J. Comput. Intell. Syst.*, vol. 17, no. 1, p. 20, Feb. 2024.



HAMID ASMAT received the Ph.D. degree in computer science from the Sarhad University of Science & Information Technology, Peshawar, Pakistan. Currently, he is a Lecturer with the Department of Information Technology, The University of Haripur, Pakistan. His research interests include information centric networking, the Internet of Things, trust management in IoT, blockchain, security in metaverse, and information security.



AYMAN ALTAMEEM received the M.Sc. degree in information systems from London South Bank University, U.K., and the Ph.D. degree in information technology from the University of Bradford, U.K. He is currently the Dean of the College of Applied Studies and Community Services, King Saud University, Riyadh. His research interests include e-commerce, the Internet of Things, information security, and artificial intelligence.



IKRAM UD DIN (Senior Member, IEEE) received the Ph.D. degree in computer science from the School of Computing, Universiti Utara Malaysia (UUM), in 2016. He was the IEEE UUM Student Branch Professional Chair. Currently, he is an Associate Professor with the Department of Information Technology, The University of Haripur. He has 15 years of teaching and research experience in different universities/organizations. His current research interests include traffic measurement and analysis for monitoring quality of service, mobility and cache management in information-centric networking, digital twins, and the Internet of Things. He has been an Associate Editor of IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, since 2022.

ment and analysis for monitoring quality of service, mobility and cache management in information-centric networking, digital twins, and the Internet of Things. He has been an Associate Editor of IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, since 2022.



AHMAD ALMOGREN (Senior Member, IEEE) received the Ph.D. degree in computer science from Southern Methodist University, Dallas, TX, USA, in 2002. He is currently a Professor with the Computer Science Department, College of Computer and Information Sciences (CCIS), King Saud University (KSU), Riyadh, Saudi Arabia, where he is also the Director of the Cyber Security Chair, CCIS. Previously, he was the Vice Dean of the Development and Quality with CCIS. He was

also the Dean of the College of Computer and Information Sciences and the Head of Academic Accreditation Council with AlYamamah University. His research interests include mobile-pervasive computing and cyber security. He served as the General Chair for the IEEE Smart World Symposium and a Technical Program Committee Member in numerous international conferences/workshops, such as IEEE CCNC, ACM BodyNets, and IEEE HPCC.



MUHAMMAD YASAR KHAN received the master's degree in computer science from the University of Peshawar, in 2008, the M.S. degree in information security from the Institute of Management Sciences, Peshawar, in 2016, and the Ph.D. degree in information technology from the University of Kuala Lumpur (UniKL), Malaysia, in 2023. He is currently a Postdoctoral Researcher with the SFI-funded E-Governance Unit, Insight Centre for Data Analytics, University of Galway,

Ireland. His research interests include privacy, trust management, android security, usage control engine, cross-domain access control management, blockchain, the Internet of Things (IoT), malware analysis, machine learning, artificial intelligence, and data analytics.

...